

CONFIDENTIEL · CONCORDE TECH

## AUDIT SÉCURITÉ & PERFORMANCE

# Concorde Workforce Rapport de remédiation

Application SaaS multi-tenant de gestion du temps —  
synthèse des vulnérabilités identifiées, correctifs apportés et  
optimisations livrées.

**Stack auditée :** ASP.NET Core 8 · EF Core 8 · SQL Server · React 18 + Vite 6 + TypeScript 5

**Périmètre :** backend ABRPOINT.Server & frontend abrpoint.client (multi-tenant master/tenant)

**Auteur :** Mohamed — Concorde Tech

**Date :** Mai 2026

# 1. Résumé exécutif

L'audit a couvert l'ensemble de la chaîne applicative — surface web (React/Vite), API ASP.NET Core (auth, tenancy, contrôleurs métier, services hostés) et persistance (EF Core, repositories, indexation SQL Server). Le périmètre incluait les flux à risque (authentification, upload, paiement Stripe, chiffrement 2FA, isolation cross-tenant) ainsi que les chemins critiques en performance (dashboard manager, état périodique, billing sync, pointage temps-réel).

45+

FINDINGS TRAITÉS

108→0

STACK TRACES FUITÉS

-65%

TAILLE BUNDLE JS

52s

BUILD (VS 1M32S)

## 1.1 Faits saillants

- **Isolation tenant durcie** — JWT enrichi de la claim `tenant_slug`, contrôle strict en middleware, élimination du bypass `/api/uploads`.
- **Cryptographie alignée** — passage AES-CBC → AES-GCM (HKDF), protection du secret TOTP côté serveur, validateur de secrets bloquant les placeholders en prod.
- **Surface XSS / mass-assignment réduite** — DTO whitelist, sanitisation SVG / DOMPurify, en-têtes COOP/COEP, suppression de `ex.Message` dans toutes les réponses HTTP.
- **Charge SGBD divisée** — cache mémoire des paramètres, indexation ciblée (10 index critiques), N+1 éliminés sur état périodique et présence.
- **Front allégé** — code splitting (React.lazy), `manualChunks` Vite, drop console/debugger en build prod, chiffrement bundle ramené à 194 Ko gzip.

## 1.2 Statut global

| Domaine                     | Findings | État                                                                                   |
|-----------------------------|----------|----------------------------------------------------------------------------------------|
| Authentification & session  | 8        | <span>Clos</span> Cookie HttpOnly, refresh rotation, captcha signup, rate-limit auth.  |
| Multi-tenant & autorisation | 6        | <span>Clos</span> Claim <code>tenant_slug</code> , route guards, uploads authentifiés. |
| Chiffrement & secrets       | 5        | <span>Clos</span> AES-GCM, HKDF, SecretsValidator, appsettings en placeholders.        |
| Entrées & uploads           | 7        | <span>Clos</span> Whitelist ext./taille, SVG strict, DOMPurify front.                  |
| Fuites d'information        | 9        | <span>Clos</span> Middleware exception globale, 108 <code>ex.Message</code> sanitisés. |
| Performance backend         | 10       | <span>Clos</span> Cache paramètres, parallel sync, batching N+1, indexation.           |
| Performance frontend        | 6        | <span>Clos</span> Lazy routes, manualChunks, esbuild drop, react-query v5 migré.       |

| Domaine                   | Findings | État                                                                                 |
|---------------------------|----------|--------------------------------------------------------------------------------------|
| Items différés (ops/data) | 7        | <span>Différé</span> Rotation secrets externes, virtualization Pointeuse, CSP nonce. |

## 2. Sommaire

---

### 1. Résumé exécutif

---

### 2. Sommaire

---

### 3. Méthodologie d'audit

---

### 4. Sécurité — Authentification, session & isolation tenant

---

### 5. Sécurité — Cryptographie & gestion des secrets

---

### 6. Sécurité — Entrées, uploads & surface XSS

---

### 7. Sécurité — Fuites d'information & durcissement réponses

---

### 8. Performance — Backend (DB, services hostés, cache)

---

### 9. Performance — Frontend (bundle, rendu, requêtes)

---

### 10. Mesures & résultats

---

### 11. Items différés & recommandations

---

### 12. Annexe — Inventaire des modifications

---

### 3. Méthodologie d'audit

L'analyse a été menée en revue de code exhaustive (white-box) avec corrélation runtime sur le serveur de développement. Chaque finding documenté est rattaché à une référence applicative (contrôleur, repository, hook React) et à une preuve d'impact (chemin d'exploitation pour le volet sécurité, mesure ou trace SQL pour le volet performance).

#### 3.1 Sources d'évidence

- Lecture du code source backend (~210 contrôleurs, services, repositories) et frontend (~430 composants React, hooks et services).
- Inspection des artefacts de build : `vite build --report` , `dotnet publish` , métriques de bundle, plans d'exécution SQL.
- Tracing applicatif : journalisation Kestrel, traçage EF Core (sensitiveDataLogging en dev), DevTools du navigateur (waterfall, performance frame, mémoire).
- Tests d'intrusion ciblés en environnement de dev (XSS reflété/persistant, IDOR cross-tenant, mass-assignment, path traversal upload, énumération de comptes via signup).

#### 3.2 Échelle de sévérité

| Niveau   | Critère                                                    | Délai de remédiation |
|----------|------------------------------------------------------------|----------------------|
| Critique | Compromission probable de données ou contournement d'auth. | Immédiat (≤ 24 h)    |
| Élevée   | Impact métier important, exploitation requérant un compte. | ≤ 1 semaine          |
| Moyenne  | Dégradation utilisateur, exposition partielle.             | ≤ 1 mois             |
| Faible   | Bonne pratique, défense en profondeur.                     | Trimestre courant    |

## 4. Sécurité — Authentification, session & isolation tenant

### S01 Critique Corrigé Cookie d'authentification accessible en JavaScript

Le cookie JWT principal était positionné sans le flag `HttpOnly`, permettant à tout script chargé dans la page (XSS, dépendance compromise, extension navigateur tiers) de l'exfiltrer.

#### AVANT

```
Response.Cookies.Append("token", jwt,  
    new CookieOptions { Secure = true });
```

#### APRÈS

```
Response.Cookies.Append("token", jwt,  
    new CookieOptions {  
        HttpOnly = true,  
        Secure = true,  
        SameSite = SameSiteMode.Strict,  
        Expires = expires  
    });
```

**Fichier :** `UtilisateursController.cs` (login, login-admin, refresh, logout).

### S02 Critique Corrigé Risque d'IDOR cross-tenant via slug manipulable

La résolution de tenant s'effectuait via header `X-Tenant-Slug` ou sous-domaine sans corrélation avec la session. Un utilisateur authentifié pouvait, en modifiant le header, consulter les écrans d'un autre tenant.

**Remédiation :** ajout de la claim `tenant_slug` dans le JWT, contrôle strict en middleware (rejet 403 si divergence claim ↔ slug courant). Le slug n'est plus autoritaire — la claim signée l'est.

```
// TenantResolverMiddleware.cs – extrait  
var claim = user.FindFirst("tenant_slug")?.Value;  
if (!string.IsNullOrEmpty(claim) &&  
    !string.Equals(claim, resolved.Slug, StringComparison.OrdinalIgnoreCase))  
{  
    ctx.Response.StatusCode = StatusCodes.Status403Forbidden;  
    return;  
}
```

### S03 Élevée Corrigé Endpoints `/api/uploads` en bypass d'authentification

Les fichiers uploadés (justificatifs, signatures, photos d'employés) étaient servis sans contrôle d'authentification ni d'isolation tenant. Toute personne devinant un nom de fichier pouvait y accéder.

**Remédiation :** nouveau contrôleur `UploadsController` `[Authorize]`, anti-traversal sur le nom, streaming via `FileStreamResult`, retrait de `/api/uploads` de la liste de bypass du middleware tenant.

### S04 Élevée Corrigé Énumération de comptes via signup & reset password

Les endpoints publics renvoyaient des codes/messages distincts selon que l'email existait ou non, permettant un user enumeration.

**Remédiation :** messages génériques systématiques, rate-limit dédié `public-form` (5 requêtes / minute / IP), captcha déclenché au-delà.

**S05** Moyenne Corrigé Champ `utiadm` persisté côté client

L'AuthProvider stockait `utiadm` dans `localStorage`, créant un vecteur d'escalade trivial via la console DevTools (l'application lisait la valeur côté front pour afficher/masquer les routes admin).

**Remédiation** : retrait de `utiadm` de `persistedKeys`, lecture exclusivement depuis le JWT (claim) au moment de l'authentification ; `RequireAdmin` route guard ajouté en complément.

**S06** Moyenne Corrigé Plan trial : accès aux fonctions premium côté front

Le hook `planAllows` retournait `true` par défaut quand `planFeatures` n'était pas encore chargé, ouvrant temporairement les écrans premium avant le contrôle serveur.

**Remédiation** : bascule en fail-closed ( `return false` si `planFeatures == null` ), contrôle serveur déjà présent en filet de sécurité.

**S07** Moyenne Corrigé Refresh token non rotatif

Le refresh token n'était pas régénéré à chaque échange, allongeant la fenêtre d'usage en cas de vol.

**Remédiation** : rotation systématique (rolling refresh), invalidation côté serveur du précédent token, révocation explicite au logout.

**S08** Faible Corrigé HSTS & HTTPS redirection en environnement dev

Activés indistinctement, ils interféraient avec le serveur de dev (Vite proxy en HTTP).

**Remédiation** : ces middlewares sont désormais conditionnels à `app.Environment.IsProduction()`.

## 5. Sécurité — Cryptographie & gestion des secrets

### S09 Critique Corrigé Chiffrement AES-CBC sans authentification

Le service `EncryptionService` utilisait AES-CBC sans MAC ; sensibles aux attaques à oracle de padding et à l'altération silencieuse du chiffré.

**Remédiation** : nouveau format AES-GCM ( `"v2:"` + `nonce[12]` || `cipher` || `tag[16]`). Une branche de compatibilité tolère le format CBC historique pour ne pas casser les données existantes pendant la migration.

```
// EncryptionService.cs – schéma v2
var nonce = RandomNumberGenerator.GetBytes(12);
var tag = new byte[16];
var cipher = new byte[plaintext.Length];
using var aes = new AesGcm(key, tagSizeInBytes: 16);
aes.Encrypt(nonce, plaintext, cipher, tag);
return "v2:" + Convert.ToBase64String(Combine(nonce, cipher, tag));
```

### S10 Critique Corrigé Secret TOTP stocké en clair

Le secret partagé du 2FA était enregistré tel quel dans la base ; toute compromission de la BD donnait accès à la 2FA de l'ensemble des utilisateurs.

**Remédiation** : service dédié `TwoFactorSecretProtector`, clé dérivée par HKDF-SHA256 depuis `Encryption:AesKey`, format `"2fa:v1:"` + `base64(nonce[12] || cipher || tag[16])`. Migration progressive : lecture transparente du legacy, ré-encodage à la première utilisation réussie.

### S11 Élevée Corrigé Secrets en clair dans `appsettings.json`

Connection strings, clés JWT, secrets Stripe et OVH versionnés en clair.

**Remédiation** : remplacement par des placeholders `REPLACE_WITH_*`, lecture obligatoire via variables d'environnement, ajout d'un `SecretsValidator` qui refuse le boot en production si un placeholder n'a pas été substitué.

### S12 Moyenne Corrigé Pas de vérification des mots de passe compromis

Les utilisateurs pouvaient choisir des mots de passe figurant dans les corpus HavelBeenPwned.

**Remédiation** : service `PasswordBreachCheckService` appelant l'API HIBP en k-anonymity (5 caractères du SHA-1, comparaison locale), résultats cachés en `IMemoryCache` 1 h pour limiter les appels sortants.



## 6. Sécurité — Entrées, uploads & surface XSS

### S13 Élevée Corrigé Mass-assignment sur création d'utilisateur

`AddUtilisateur` liait directement le modèle EF reçu en POST, exposant les colonnes `Utiroleid`, `Utisuper`, `Utitenantid`.

**Remédiation** : DTO whitelist `CreateUtilisateurDto` (champs autorisés explicitement), mapping manuel vers l'entité, valeurs sensibles dérivées côté serveur.

### S14 Élevée Corrigé XSS persistant via upload SVG

Le pad de signature acceptait des images base64 (PNG côté web, SVG côté mobile). L'extension n'était pas inférée correctement : un SVG portant `<script>` ou `onload` stocké sous `.png` pouvait être servi sur la même origine et exécuté lors de la prévisualisation.

**Remédiation** : détection MIME stricte côté `FileHelper.SaveBase64Image`, validation SVG en parsing XML (whitelist d'éléments `path` / `polyline` / `line` ...), rejet de tout attribut `on*` ou URL `javascript:` / `data:text/html`.

```
// FileHelper.cs - IsSafeSvg (extrait)
if (attr.StartsWith("on", StringComparison.OrdinalIgnoreCase))
    return reason = "attribut événement interdit", false;
if (value.StartsWith("javascript:", StringComparison.OrdinalIgnoreCase))
    return reason = "URL suspecte", false;
```

### S15 Élevée Corrigé Upload sans whitelist d'extension ni plafond de taille

Tout type de fichier était accepté (`.aspx`, `.php`, `.exe`), sans limite de taille — saturation disque possible.

**Remédiation** : whitelist `AllowedExtensions` (PDF, images courantes, bureautique), nom de fichier régénéré en UUID (anti path-traversal et double-extension), plafond uniforme 10 Mo paramétrable par `Uploads__MaxSizeMb`.

### S16 Moyenne Corrigé HTML riche injecté sans sanitisation (constructeur de contrats)

4 sites utilisaient `dangerouslySetInnerHTML` ou `element.innerHTML` sur du contenu utilisateur (templates de contrats, signature de modèle).

**Remédiation** : utilitaire `sanitizeRichHtml` basé sur DOMPurify, appliqué systématiquement avant injection. Promu en dépendance directe et retiré de `overrides`.

### S17 Moyenne Corrigé Signup sans captcha

L'inscription publique permettait la création automatisée de comptes.

**Remédiation** : captcha intégré au formulaire de signup, vérification serveur, couplée au rate-limit `public-form`.

## 7. Sécurité — Fuites d'information & durcissement réponses

**S18** Élevée Corrigé 108 occurrences de `ex.Message` renvoyées au client

L'inventaire a relevé 108 retours HTTP contenant `ex.Message` (avec parfois la stack trace dans `Details`). Risque : exposition de noms de tables, contraintes uniques, chemins absolus, secrets de configuration partiellement résolus.

**Remédiation** : trois passes :

1. Substitution automatisée (PowerShell + `UTF8Encoding($false)` pour éviter le BOM) sur les patterns courants : `details`, `error`, `Message`, `BadRequest(ex.Message)`, etc.
2. Passe complémentaire sur les variantes camelCase et interpolations.
3. Passe manuelle ciblée sur les cas restants ( `UtilisateursController` lignes 657/678/696, `FileHelper.SaveBase64Image`, `EmployeeRepository.DeleteEmployeeAsync` ).

Le logger serveur ( `_logger.LogError(ex, ...)` ) est systématiquement préservé. Message client uniforme : « *Erreur interne. Consultez les logs serveur pour le détail.* »

**S19** Élevée Corrigé Pas de filet d'exception global

Une exception non gérée dans un contrôleur pouvait remonter une page d'erreur ASP.NET détaillée (en dev) ou un body vide (en prod) sans corrélation avec les logs.

**Remédiation** : middleware `GlobalExceptionHandler` : logs serveur incluant le `TraceIdentifier`, réponse HTTP 500 générique au client référençant ce `correlationId` pour le support.

**S20** Moyenne Corrigé Source maps front livrées en production

Les `.map` étaient générées en build de production, exposant le source TS et les éventuels secrets inlinés.

**Remédiation** : `build.sourcemap = false` dans `vite.config.ts` ; `esbuild.drop = ['console', 'debugger']` en prod pour éliminer les traces de debug résiduelles.

**S21** Faible Corrigé En-têtes de sécurité incomplets

Absence de `Strict-Transport-Security` en prod, X-Content-Type-Options inconstant.

**Remédiation** : HSTS, X-Content-Type-Options, Referrer-Policy ajoutés en pipeline. CSP nonce-based listée pour un sprint dédié (cf. items différés).

## 8. Performance — Backend (DB, services hostés, cache)

### P01 Élevée Corrigé Lecture des paramètres tenant à chaque requête

`ParametreRepository` faisait une lecture SQL à chaque appel — appelée des dizaines de fois par requête utilisateur via les helpers métier (arrondi, périodes, devises).

**Remédiation :** `GetParametreCachedAsync` backé par `IMemoryCache` (TTL 5 min, clé tenant-scoped), invalidation explicite sur Add/Update/Delete.

### P02 Élevée Corrigé N+1 sur l'état périodique (employé × mois)

Pour chaque employé du périmètre, un SELECT distinct par mois : sur un export annuel multi-employés, plusieurs centaines de requêtes étaient émises.

**Remédiation :** nouvelle méthode `GetEmpparamBatchAsync` qui charge l'ensemble en 3 SELECT (paramètres employé, calendrier, présences) puis assemble en mémoire. Sur un export 50 employés × 12 mois : ~600 requêtes → 3.

### P03 Élevée Corrigé Recompilation systématique des `DbContextOptions` par tenant

La factory tenant reconstruisait les options EF Core à chaque résolution — coût d'analyse du provider et du model snapshot.

**Remédiation :** cache statique `ConcurrentDictionary<string, DbContextOptions>`, clé = connection string. ApplicationDbContext (master) bénéficie du même cache via `_dbOptionsCache` dans `Program.cs`.

### P04 Moyenne Corrigé Synchronisation billing séquentielle

Le service hosté `EmployeeBillingSyncService` traitait les tenants en boucle `foreach await`, sérialisant les appels Stripe.

**Remédiation :** `Parallel.ForEachAsync` avec `MaxDegreeOfParallelism = 8`, compteurs `Interlocked.Increment` pour la télémétrie. Idem pour `PunctualityReminderHostedService` (degree=4).

### P05 Moyenne Corrigé Listes employés/libellés non plafonnées et trackées

`GetAllAsync` et `GetEmpLibs` chargeaient l'intégralité de la table en tracking EF — saturation mémoire sur tenant 5k+ employés.

**Remédiation :** `AsNoTracking()` systématique, plafonds (2000 pour `GetAllAsync`, 5000 pour `GetEmpLibs`) avec instrumentation d'alerte si atteint.

### P06 Moyenne Corrigé Dashboard manager : 6 `CountAsync` distincts

`ManagerDashboardController` émettait 6 round-trips SQL pour calculer les KPI d'accueil.

**Remédiation :** projection unique sur `Societes` agrégeant les 6 indicateurs en un `SELECT ... COUNT(CASE WHEN ...)`.

**P07** Moyenne Corrigé Indexation manquante sur les tables chaudes

Les plans d'exécution montraient des scans complets sur `Presences`, `Pointages`, `Employes`, `DemConges`.

**Remédiation :** méthode `EnsurePerformanceIndexesAsync` exécutée au démarrage, créant 10 index ciblés (clés composites tenant + date / tenant + statut / tenant + employé selon le cas) si absents.

**P08** Faible Corrigé Téléchargement Vault via `MemoryStream` intermédiaire

Le contrôleur Vault chargeait le fichier entier en mémoire avant de l'écrire dans la réponse — pression GC sur les pièces volumineuses.

**Remédiation :** `FileStreamResult` direct avec `FileStream` ouvert en lecture séquentielle.

**P09** Faible Corrigé `async void` sur signature de repository

`PresenceRepository.Update` était déclaré `async void` — les exceptions remontaient en `UnobservedTaskException`.

**Remédiation :** conversion en `async Task`, callers attendent.

**P10** Faible Corrigé Imports en masse non groupés

`BulkImportController` faisait un `SaveChanges` par ligne, multipliant les round-trips.

**Remédiation :** `BatchCodeGenerator` + un seul `SaveChangesAsync` par lot ; gestion d'erreurs sanitisée ( `"{lib}: erreur interne"` au lieu de l'exception brute).

## 9. Performance — Frontend (bundle, rendu, requêtes)

### P11 Élevée Corrigé Bundle JS monolithique de 1,99 Mo (537 Ko gzip)

Chargement intégral à l'authentification : MUI, MUI X DataGrid, jsPDF, xlsx, FullCalendar, Recharts, DOMPurify regroupés dans un seul fichier.

#### Remédiation :

- `React.lazy` sur ~60 routes pages + `<React.Suspense>` avec fallback léger.
- `build.manualChunks` Vite : `mui-core`, `mui-x`, `pdf`, `spreadsheet`, `calendar`, `charts` isolés.
- `esbuild.drop = ['console', 'debugger']`, `sourcemap: false`.

Résultat : **630 Ko / 194 Ko gzip** sur le chunk principal (–65 %), build 1m32s → 52s.

### P12 Moyenne Corrigé Re-renders globaux via AuthProvider

L'objet `value` du contexte était reconstruit à chaque render, propageant le re-render à toute la sous-arbre.

**Remédiation :** `useCallback` sur `setAuth` / `clearAuth`, mémoïsation du `value` via `useMemo` avec dépendances minimales. `clearAuth` appelle désormais `/Utilisateurs/logout` pour révoquer le refresh token serveur.

### P13 Moyenne Corrigé Liste de pointages saturée (10k+ lignes)

`PointageList` rendait l'intégralité des pointages d'une journée — gel UI sur tenant fortement actif.

**Remédiation :** plafond à 500 lignes avec bandeau « affichage tronqué », `key` stable `${employee_code}-${time}` pour limiter les reconciliations. Virtualization complète prévue en sprint dédié (cf. items différés).

### P14 Moyenne Corrigé react-query v3 / v5 cohabitation et erreurs typage

L'audit a relevé 174 hooks utilisant l'ancienne syntaxe `react-query` (v3) coexistant avec `@tanstack/react-query` v5. `tsc` remontait **144 erreurs** typées.

#### Remédiation : migration complète :

- `isLoading` → `isPending` sur les mutations (15 composants).
- `invalidateQueries(key)` → `invalidateQueries({ queryKey })` (objet obligatoire en v5).
- `cacheTime` → `gcTime`.
- Retrait de `onError` sur `useQuery` (non supporté en v5).
- Conversion signatures tuple `(key, fn, opts)` → objet `{ queryKey, queryFn, ... }` sur 5 hooks et CahierConge.
- Narrowing du type `data: never[] | TQueryFnData` via cast `X[] = dataRaw ?? []` (EmployeModern, PaysModern, DashboardModern, CahierConge).

Résultat : `npx tsc -b` → **exit 0, 0 erreurs**.

**P15** Faible Corrigé Dépendances inutilisées en bundle

Inventaire `package.json` : `dotenv` , `material-react` , `@types/react-router-dom` tirés sans usage.

**Remédiation** : suppression ; gain mineur mais utile en lockfile et audit de licences. `dompurify` promu en dépendance directe (retrait de `overrides` ).

**P16** Faible Corrigé Assets statiques non optimisés

Favicon mono-taille, OG image manquante, logo unique haute résolution chargé sur toute la nav.

**Remédiation** : set complet ( `favicon-32.png` , `icon-192.png` , `logo-256.png` , `og-image.jpg` ), références consolidées dans `index.html` .

## 10. Mesures & résultats

### 10.1 Métriques avant / après

| Indicateur                                               | Avant                             | Après  | Δ             |
|----------------------------------------------------------|-----------------------------------|--------|---------------|
| Bundle JS principal (gzip)                               | 537 Ko                            | 194 Ko | −64 %         |
| Bundle JS principal (brut)                               | 1,99 Mo                           | 630 Ko | −68 %         |
| Temps de build front                                     | 1m32s                             | 52s    | −44 %         |
| Erreurs <code>tsc</code>                                 | 144                               | 0      | −100 %        |
| Retours HTTP avec <code>ex.Message</code>                | 108                               | 0      | −100 %        |
| Requêtes SQL — export état périodique (50 emp / 12 mois) | ~600                              | 3      | −99 %         |
| SELECT dashboard manager (chargement)                    | 6                                 | 1      | −83 %         |
| Endpoints exposés sans auth                              | 1 ( <code>/api/uploads/*</code> ) | 0      | <b>résolu</b> |

### 10.2 Conformité après remédiation

| Domaine OWASP / RGPD               | Avant                                              | Après                                                    |
|------------------------------------|----------------------------------------------------|----------------------------------------------------------|
| A01 Broken Access Control          | Cross-tenant via slug, uploads anonymes            | Claim signée + middleware strict, endpoints authentifiés |
| A02 Cryptographic Failures         | AES-CBC, secrets en clair, TOTP non protégé        | AES-GCM, HKDF, placeholders + validateur secrets         |
| A03 Injection / XSS                | SVG XSS, innerHTML non sanitisé, mass-assignment   | Validation SVG, DOMPurify, DTO whitelist                 |
| A04 Insecure Design                | Pas de rate-limit signup, énumération comptes      | Rate-limit + captcha, messages uniformes                 |
| A05 Security Misconfiguration      | HSTS partout, source maps prod, secrets versionnés | HSTS prod-only, sourcemap off, secrets externalisés      |
| A07 Identification & Auth Failures | Cookie non-HttpOnly, refresh non rotatif           | HttpOnly + Secure + SameSite, rolling refresh            |
| A09 Logging & Monitoring Failures  | Stack traces au client, pas de corrélation         | GlobalExceptionHandler + TracerIdentifier                |

## 11. Items différés & recommandations

Les items ci-dessous ne sont pas bloquants mais sont planifiés sur les sprints à venir. Ils relèvent d'opérations infrastructure, d'actions data ou de refactors lourds dont le coût/risque a été jugé incompatible avec la fenêtre de remédiation immédiate.

| Item                                                                             | Type       | Estimation | Justification du report                                                                       |
|----------------------------------------------------------------------------------|------------|------------|-----------------------------------------------------------------------------------------------|
| Rotation des secrets externes (Stripe, OVH, JWT)                                 | Ops        | 0,5 j      | Coordination avec admin Stripe/OVH ; à exécuter dans la même fenêtre que le déploiement prod. |
| Migration uploads par sous-dossier tenant ( <code>/uploads/{tenant}/...</code> ) | Ops + data | 2 j        | Nécessite migration des fichiers existants + mise à jour des URL en base.                     |
| Suppression branches legacy <code>EncryptionService.Decrypt</code> (AES-CBC)     | Refacto    | 0,5 j      | À planifier après ré-encodage à 100 % des champs concernés (script de migration data).        |
| CSP nonce-based stricte ( <code>script-src 'nonce-...'</code> )                  | Sécurité   | 3 j        | Impact transverse sur les scripts inlines et les libs tierces ; sprint dédié.                 |
| Imports MUI deep paths (codemod <code>@mui/codemod</code> )                      | Perf front | 1 j        | Gain bundle estimé 30-50 Ko ; codemod officiel à appliquer et tester en intégration.          |
| Virtualization full DataGrid (Pointeuse / ListeEmploye / EmpEtatPeriodique)      | Perf front | 2 j        | Le cap à 500 lignes mitige déjà ; virtualization à activer pour tenants > 5k pointages/j.     |
| Retrait <code>material-symbols</code> (137 usages × 17 fichiers)                 | Bundle     | 1,5 j      | Substitution un-pour-un par MUI Icons ; mécanique, non risquée mais volumineuse.              |

### 11.1 Recommandations stratégiques

- **SIEM / observabilité** — corréler les `correlationId` désormais émis par le `GlobalExceptionHandler` avec un agrégateur de logs centralisé (Seq, Grafana Loki ou équivalent) pour faciliter le triage post-incident.
- **Tests d'intrusion périodiques** — programmer un pentest externe annuel une fois la CSP nonce-based déployée, idéalement après la rotation des secrets.
- **SAST / Dependabot** — activer un scan automatique des PR (CodeQL ou SonarCloud) pour prévenir la réintroduction des patterns corrigés ( `ex.Message` , `innerHTML` non sanitisé, `cacheTime` ...).
- **Plan de continuité 2FA** — documenter le chemin de récupération pour les utilisateurs ayant perdu leur secret TOTP désormais chiffré (la base seule ne suffit plus à le récupérer, ce qui est l'effet recherché).



## 12. Annexe — Inventaire des modifications

### 12.1 Fichiers backend créés / modifiés

| Fichier                                                      | Nature                                                                      |
|--------------------------------------------------------------|-----------------------------------------------------------------------------|
| ABRPOINT.Server/Controllers/UtilisateursController.cs        | Cookie HttpOnly, claim tenant_slug, DTO whitelist, sanitisation ex.Message. |
| ABRPOINT.Server/Controllers/BulkImportController.cs          | BatchCodeGenerator, sanitisation erreurs.                                   |
| ABRPOINT.Server/Controllers/ManagerDashboardController.cs    | 6 CountAsync → 1 projection.                                                |
| ABRPOINT.Server/Controllers/UploadsController.cs             | Nouveau — endpoint [Authorize] + streaming.                                 |
| ABRPOINT.Server/Controllers/VaultController.cs               | FileStreamResult direct.                                                    |
| ABRPOINT.Server/Tenancy/TenantResolverMiddleware.cs          | Contrôle strict claim tenant_slug, retrait bypass uploads.                  |
| ABRPOINT.Server/Tenancy/ITenantDbContextFactory.cs           | Cache statique des DbContextOptions.                                        |
| ABRPOINT.Server/Services/EncryptionService.cs                | AES-GCM v2, fallback CBC tolérant.                                          |
| ABRPOINT.Server/Services/TwoFactorSecretProtector.cs         | Nouveau — HKDF + AES-GCM, format 2fa:v1:.                                   |
| ABRPOINT.Server/Services/PasswordBreachCheckService.cs       | HIBP k-anonymity + IMemoryCache.                                            |
| ABRPOINT.Server/Services/BaseDataSchemaMigrator.cs           | EnsurePerformanceIndexesAsync (10 index).                                   |
| ABRPOINT.Server/Billing/EmployeeBillingSyncService.cs        | Parallel.ForEachAsync degree=8.                                             |
| ABRPOINT.Server/Services/PunctualityReminderHostedService.cs | Parallel.ForEachAsync degree=4.                                             |
| ABRPOINT.Server/Middleware/GlobalExceptionHandler.cs         | Nouveau — TracelIdentifier + log + 500 générique.                           |
| ABRPOINT.Server/Helpers/FileHelper.cs                        | Whitelist + IsSafeSvg + plafond paramétrable.                               |
| ABRPOINT.Server/Helpers/SecretsValidator.cs                  | Nouveau — refus boot prod sur placeholders.                                 |
| ABRPOINT.Server/Helpers/SequentialCodeGenerator.cs           | BatchCodeGenerator.                                                         |
| ABRPOINT.Server/Repository/EmployeeRepository.cs             | GetEmpparamBatchAsync, caps + AsNoTracking, sanitisation.                   |
| ABRPOINT.Server/Repository/ParametreRepository.cs            | GetParametreCachedAsync + invalidation.                                     |

| Fichier                                                       | Nature                                                        |
|---------------------------------------------------------------|---------------------------------------------------------------|
| <code>ABRPOINT.Server/Repository/PresenceRepository.cs</code> | async void → async Task.                                      |
| <code>ABRPOINT.Server/Dtaos/CreateUtilisateurDto.cs</code>    | <i>Nouveau</i> — DTO whitelist anti mass-assignment.          |
| <code>ABRPOINT.Server/Program.cs</code>                       | Cache options, HSTS prod, rate-limit, GlobalExceptionHandler. |
| <code>ABRPOINT.Server/appsettings.json</code>                 | Tous secrets → REPLACE_WITH_*.                                |

## 12.2 Fichiers frontend créés / modifiés

| Fichier                                                                                    | Nature                                                                                  |
|--------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------|
| <code>abrpoint.client/src/components/navigation/Navigation.tsx</code>                      | React.lazy + Suspense, useMemo, RouteGuards.                                            |
| <code>abrpoint.client/src/components/helper/AuthProvider.tsx</code>                        | useCallback / useMemo, fail-closed planAllows, retrait utiadm persisté, logout serveur. |
| <code>abrpoint.client/src/components/helper/RouteGuards.tsx</code>                         | <i>Nouveau</i> — RequireAuth, RequireAdmin, classifyRoute.                              |
| <code>abrpoint.client/src/components/helper/sanitizeHtml.ts</code>                         | <i>Nouveau</i> — wrapper DOMPurify.                                                     |
| <code>abrpoint.client/src/components/gestionEmploye/Vault/ContractBuilderModern.tsx</code> | 4 sites innerHTML sanitisés.                                                            |
| <code>abrpoint.client/vite.config.ts</code>                                                | manualChunks, sourcemap off, esbuild drop.                                              |
| <code>abrpoint.client/index.html</code>                                                    | Set d'icons complet, OG image.                                                          |
| <code>abrpoint.client/package.json</code>                                                  | Suppression deps inutilisées, DOMPurify promu.                                          |
| <code>abrpoint.client/src/components/Pointeuse/Lecture/PointageList.tsx</code>             | Cap 500 + key stable.                                                                   |
| 174 hooks <code>src/hooks/</code>                                                          | Migration <code>react-query</code> →                                                    |

| Fichier                                                           | Nature                                                                      |
|-------------------------------------------------------------------|-----------------------------------------------------------------------------|
|                                                                   | @tanstack/react-query v5.                                                   |
| 15 composants (FilialeModern, SocieteModern, DemCongeModern, ...) | isLoading →<br>isPending,<br>invalidateQueries<br>objet, narrowing<br>data. |

Rapport généré sur la base d'une revue exhaustive du dépôt Git **GestTemps** (branche **master** ), avec validation **dotnet build** (0 erreur) et **npx tsc -b** (0 erreur, 0 avertissement bloquant) en clôture de chaque lot de remédiation. Les commits associés sont disponibles dans l'historique du dépôt et peuvent être annotés sur demande.